

Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Para la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso como, por ejemplo, los mapas de Karnaugh o los sistemas de representación. Por lo tanto, se recomienda que le deis un vistazo.

Competencias

Conocer la organización general de un computador como circuito digital y conocer las características distintivas de la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la elección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

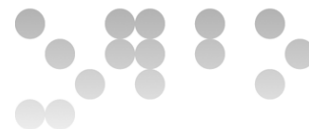
Recursos

Los recursos que se recomienda usar para esta práctica son los siguientes:

- **Básicos:** El módulo 5 de los materiales.
- **Complementarios:** VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas no justificadas no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.



Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas publicadas junto con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **22 de diciembre**, a las 24 horas.

Solución de la práctica propuesta

PRIMERA PARTE [65%]

Dada la máquina de estados siguiente:

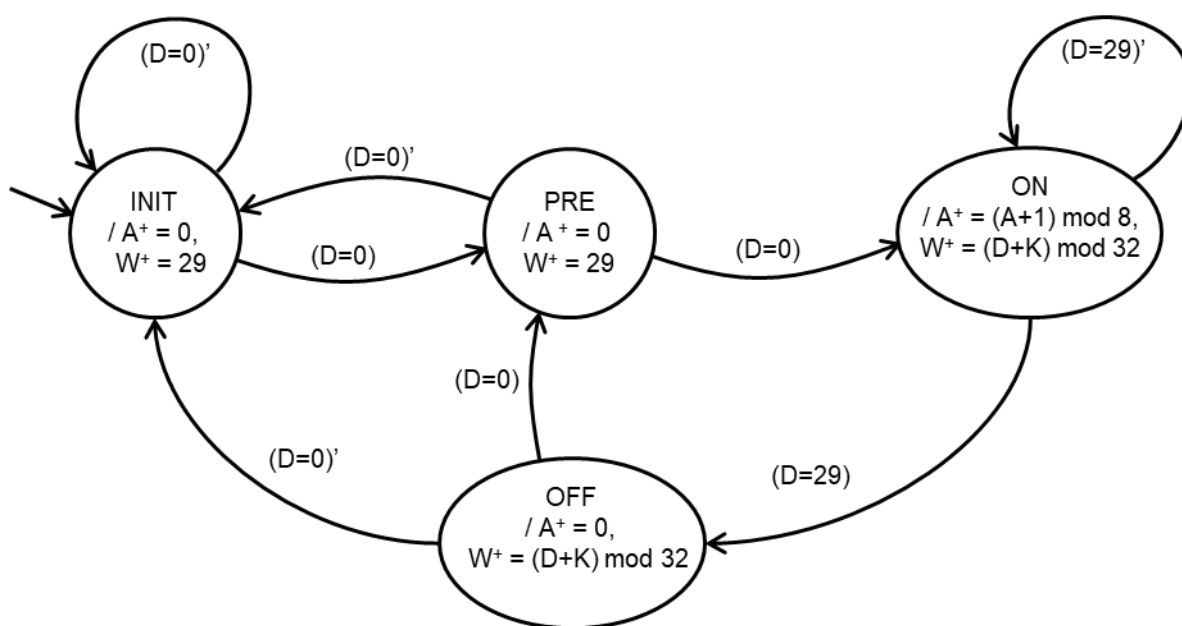
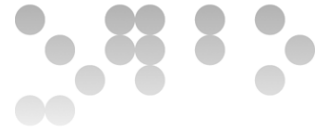


Fig. 1. EFSM de un encriptador.

Se pide que:

- a) [15%] A partir del grafo de la EFSM de la fig. 1, completad en el cronograma siguiente la evolución de las señales $(D=0)$ y $(D=29)$, de las variables W y A , y del estado (E) en que se encuentra la máquina en cada ciclo de reloj.

Nota: Tenéis el ejercicio disponible en VerilChart, donde el estado es E y los valores de D (5 bits), de A (3 bits), de K (5 bits) y de W (5 bits), se representan en hexadecimal.

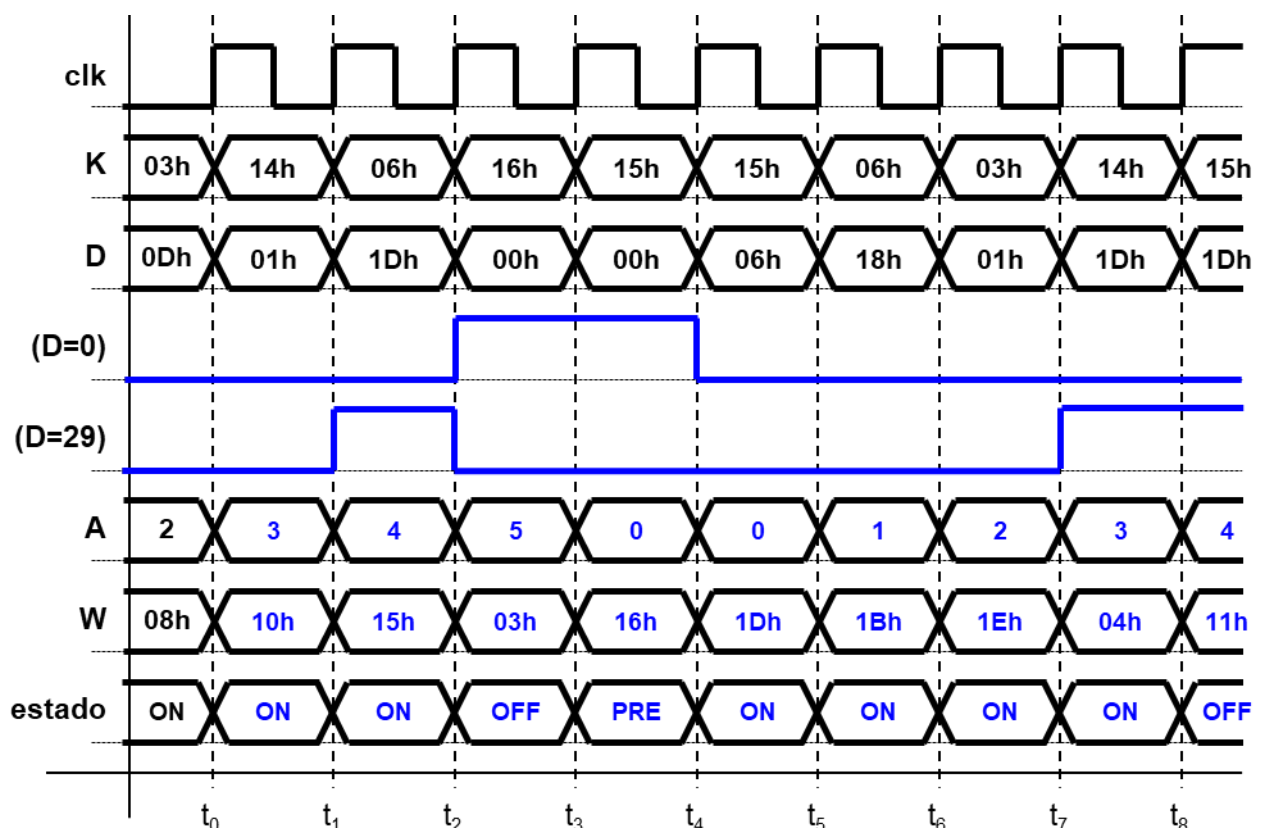


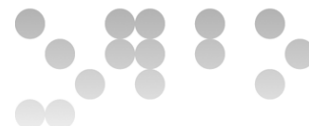
El valor de las señales ($D=0$) y ($D=29$) depende de forma exclusiva y combinacional del valor de la variable D . El nuevo estado se determina a partir del estado actual y de las señales ($D=0$) y ($D=29$). En las condiciones iniciales, el estado es ON y la variable D tiene el valor 0Dh. Por lo tanto, ($D=0$) y ($D=29$) son 0 y, consiguientemente, el nuevo estado será ON.

A partir del estado en las condiciones iniciales, ON, se pueden determinar los valores de las variables A y W para el periodo t_0-t_1 , teniendo en cuenta los valores iniciales de K , D y A .

Por lo tanto, en cada ciclo se examina el valor de la variable D para determinar ($D=0$) y ($D=29$). Con estas señales y el estado actual se determina el estado en el siguiente ciclo. Y a partir del estado actual y de los valores que tengamos en K , D y A , se determina el valor de las variables A y W en el ciclo siguiente.

Con este procedimiento, se pueden determinar los valores para el periodo t_1-t_2 a partir de los del periodo anterior, y se puede ir repitiendo el procedimiento hasta completar el cronograma, tal como se muestra en la figura siguiente.





- b) [15%] Obtened las tablas de transiciones y de salidas de la EFSM de la fig. 1, así como las codificaciones binarias de estados y salidas correspondientes.

En el grafo hay dos señales de entrada: ($D=0$) y ($D=29$). Además, tenemos las variables A y W para las que se tiene que determinar qué valores toman.

Las tablas de transiciones y salidas que se obtienen a partir del grafo son:

Estado actual	Entrada	Estado siguiente	Estado	Salidas
INIT	($D=0$)'	INIT	INIT	$A^+ = 0, W^+ = 29$
INIT	($D=0$)	PRE	PRE	$A^+ = 0, W^+ = 29$
PRE	($D=0$)'	INIT	ON	$A^+ = (A+1) \bmod 8, W^+ = (D+K) \bmod 32$
PRE	($D=0$)	ON	OFF	$A^+ = 0, W^+ = (D+K) \bmod 32$
ON	($D=29$)'	ON		
ON	($D=29$)	OFF		
OFF	($D=0$)'	INIT		
OFF	($D=0$)	PRE		

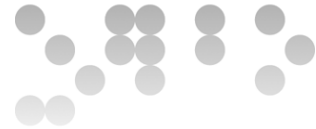
Para la codificación binaria, hay que asignar a cada estado un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde el 0 para el estado inicial hasta el número de estados que haya menos 1: 00 para INIT, 01 para PRE, 10 para ON y 11 para OFF.

Para elegir el valor siguiente que se asigna a la variable A hace falta un selector de 1 bit, dado que solo hay dos posibles valores de A , y que denominaremos s_A . En la tabla siguiente tenemos los valores entre los que se escoge y una posible codificación:

Operación	Selector s_A	Efecto sobre la variable
$A^+ = 0$	0	Ponerla a 0
$A^+ = (A+1) \bmod 8$	1	Incrementarla en 1 y aplicar módulo 8 al resultado

De forma similar, necesitaremos un selector, s_W , para elegir uno de los dos posibles valores de la variable W . En la tabla siguiente se muestran los valores y una posible codificación binaria:

Operación	Selector s_W	Efecto sobre la variable
$W^+ = 29$	0	Ponerla a 29
$W^+ = (D+K) \bmod 32$	1	Sumarle K y aplicar módulo 32 al resultado

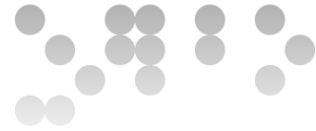


Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondiente son:

Estado actual		Entrada		Estado ⁺
Símbolo	q_1q_0	(D=0)	(D=29)	$q_1^+q_0^+$
INIT	00	0	x	00
INIT	00	1	x	01
PRE	01	0	x	00
PRE	01	1	x	10
ON	10	x	0	10
ON	10	x	1	11
OFF	11	0	x	00
OFF	11	1	x	01

Estado		Selectores	
Símbolo	q_1q_0	s_A	s_W
INIT	00	0	0
PRE	01	0	0
ON	10	1	1
OFF	11	0	1

Notad que el valor del selector s_W coincide con el valor del bit de más peso del código de estado, q_1 . Por lo tanto, podríamos usar q_1 directamente como selector para los valores de la variable W .



La EFSM siguiente muestra el comportamiento del controlador de un módulo contador:

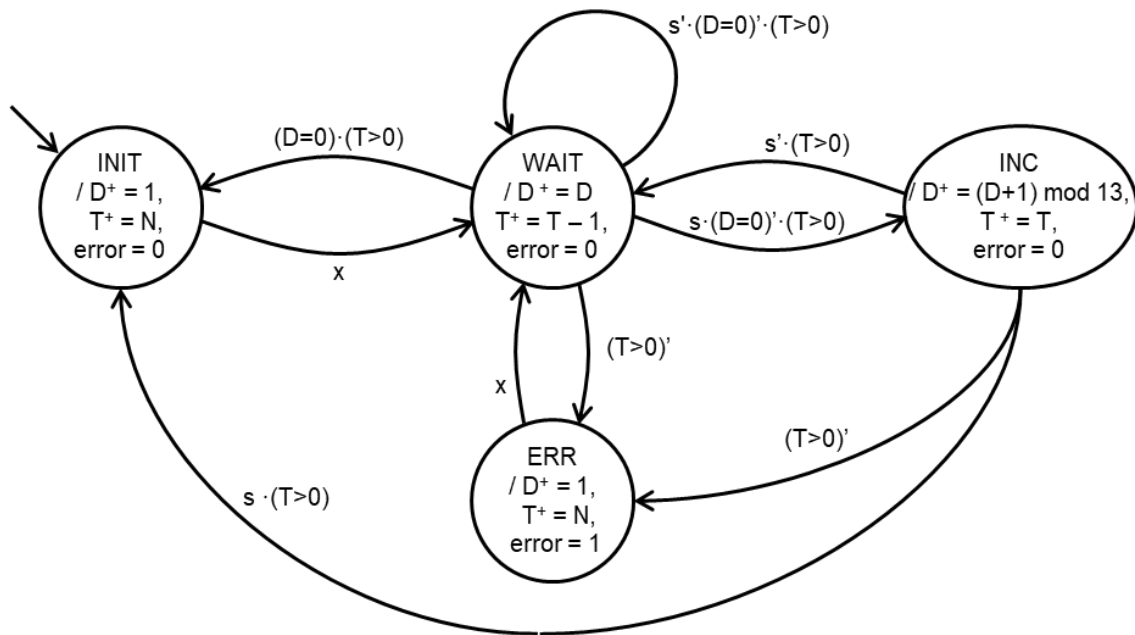


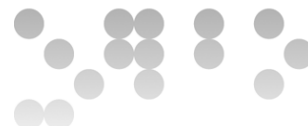
Fig. 2. EFSM del controlador de un módulo contador.

- c) [15%] A partir de las tablas de transiciones y de salidas del grafo de la fig. 2 que se dan a continuación, implementad la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creáis convenientes. Especificad y justificad las dimensiones de la/las ROM que uséis e indicad el contenido en binario, indicando a que corresponde cada bit, y en hexadecimal.

Estado actual		Entradas			Estado ⁺
Símbolo	q ₁ q ₀	s	(D=0)	(T>0)	q ₁ ⁺ q ₀ ⁺
INIT	00	x	x	x	01
WAIT	01	x	1	1	00
WAIT	01	0	0	1	01
WAIT	01	1	0	1	10
WAIT	01	x	x	0	11
INC	10	1	x	1	00
INC	10	0	x	1	01
INC	10	x	x	0	11
ERR	11	x	x	x	01

Estado		Salidas		
Símbolo	q ₁ q ₀	s_D	s_T	error
INIT	00	00	00	0
WAIT	01	01	10	0
INC	10	10	01	0
ERR	11	00	00	1

Nota: Tenéis el ejercicio disponible en VerilCIRC y, si queréis comprobar previamente el correcto funcionamiento de la ROM, la tenéis disponible en VerilChart. En este entorno, el estado es E, (T>0) es T0 y (D=0) es D0.

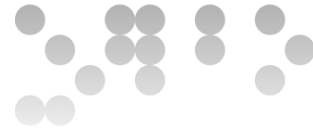


Para implementar la unidad de control con una ROM hace falta, primero, determinar el tamaño y, después, calcular el contenido a partir de la tabla de transiciones.

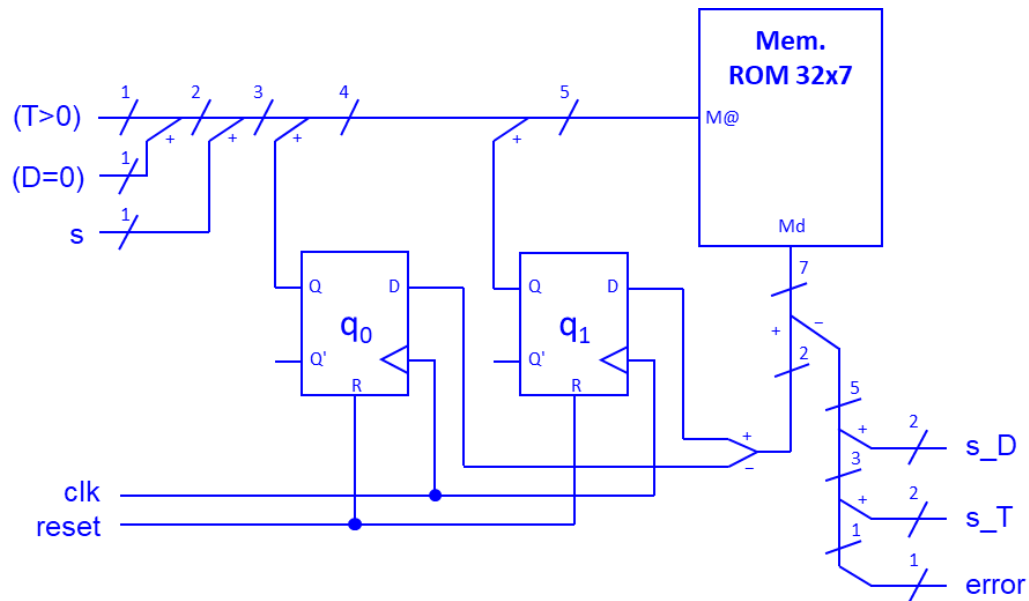
El bus de direcciones de la ROM debe ser de 5 bits, ya que la dirección se forma con los bits que codifican el estado (2 bits) y los de las señales de entrada (3 más). Por lo tanto, la ROM debe disponer de un total de $2^5 = 32$ posiciones de memoria. En cada posición se almacena el código del estado siguiente y el valor, en el estado actual, de cada una de las señales de salida. Por lo tanto, hacen falta 2 bits para el estado y 5 bits para las salidas (s_D , s_T y $error$), lo que hace un total de 7 bits.

Con todo, la ROM tiene que ser de 32 palabras de 7 bits. El contenido de la ROM se determina partir de la tabla de transiciones:

Posición de memoria				Contenido	Codificación hexadecimal
q_1q_0	s	(D=0)	(T>0)	$q_1^+ q_0^+ s_D s_T error$	
00	0	0	0	01 00 00 0	20h
00	0	0	1	01 00 00 0	20h
00	0	1	0	01 00 00 0	20h
00	0	1	1	01 00 00 0	20h
00	1	0	0	01 00 00 0	20h
00	1	0	1	01 00 00 0	20h
00	1	1	0	01 00 00 0	20h
00	1	1	1	01 00 00 0	20h
01	0	0	0	11 01 10 0	6Ch
01	0	0	1	01 01 10 0	2Ch
01	0	1	0	11 01 10 0	6Ch
01	0	1	1	00 01 10 0	0Ch
01	1	0	0	11 01 10 0	6Ch
01	1	0	1	10 01 10 0	4Ch
01	1	1	0	11 01 10 0	6Ch
01	1	1	1	00 01 10 0	0Ch
10	0	0	0	11 10 01 0	72h
10	0	0	1	01 10 01 0	32h
10	0	1	0	11 10 01 0	72h
10	0	1	1	01 10 01 0	32h
10	1	0	0	11 10 01 0	72h
10	1	0	1	00 10 01 0	12h
10	1	1	0	11 10 01 0	72h
10	1	1	1	00 10 01 0	12h
11	0	0	0	01 00 00 1	21h
11	0	0	1	01 00 00 1	21h
11	0	1	0	01 00 00 1	21h
11	0	1	1	01 00 00 1	21h
11	1	0	0	01 00 00 1	21h
11	1	0	1	01 00 00 1	21h
11	1	1	0	01 00 00 1	21h
11	1	1	1	01 00 00 1	21h

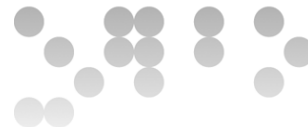


La figura siguiente muestra la implementación con una ROM de estas dimensiones. La entrada $M@$ de la ROM se configura con el valor del estado actual (los 2 bits de más peso) y el valor de las entradas (los 3 bits de menos peso). Para guardar el estado actual se usan 2 biestables.



La otra opción es usando dos ROM, una para el cálculo del estado siguiente y la otra para las salidas. En este caso, la ROM de cálculo del estado siguiente tiene el mismo número de bits de dirección que el anterior (es decir, 5) pero sólo dos bits para el contenido, puesto que sólo se almacena el código del estado siguiente:

Posición de memoria				Contenido	Codificación hexadecimal
q_1q_0	s	(D=0)	(T>0)	$q_1^+ q_0^+$	
00	0	0	0	01	1h
00	0	0	1	01	1h
00	0	1	0	01	1h
00	0	1	1	01	1h
00	1	0	0	01	1h
00	1	0	1	01	1h
00	1	1	0	01	1h
00	1	1	1	01	1h
01	0	0	0	11	3h
01	0	0	1	01	1h
01	0	1	0	11	3h
01	0	1	1	00	0h
01	1	0	0	11	3h
01	1	0	1	10	2h
01	1	1	0	11	3h
01	1	1	1	00	0h
10	0	0	0	11	3h
10	0	0	1	01	1h

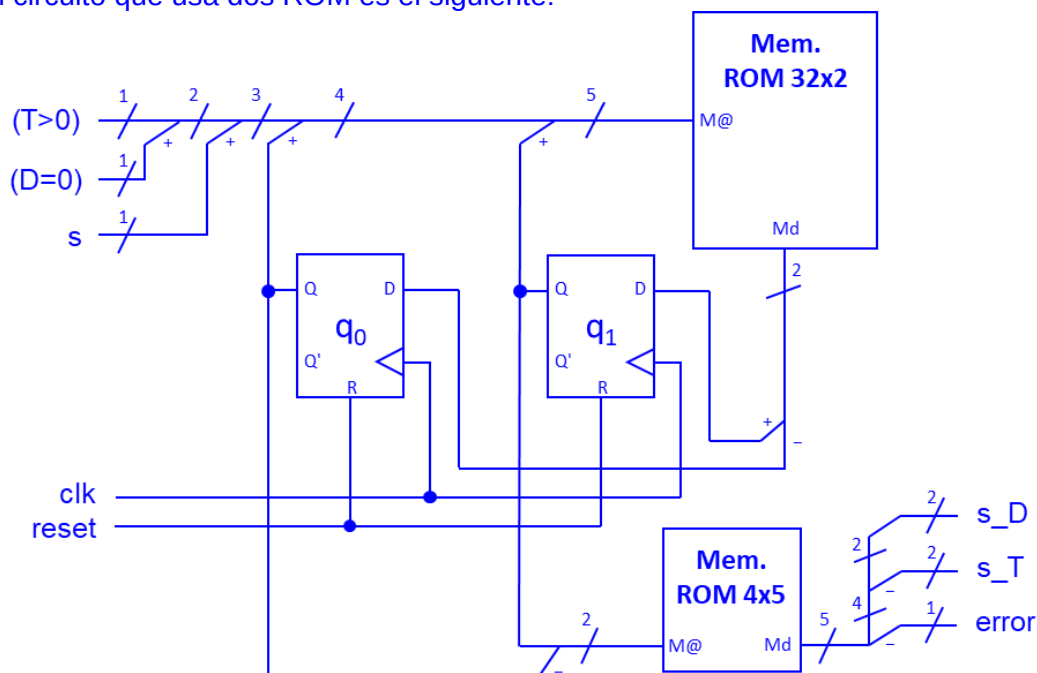


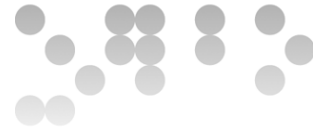
Posición de memoria				Contenido	Codificación hexadecimal
q_1q_0	s	(D=0)	(T>0)	$q_1^+ q_0^+$	
10	0	1	0	11	3h
10	0	1	1	01	1h
10	1	0	0	11	3h
10	1	0	1	00	0h
10	1	1	0	11	3h
10	1	1	1	00	0h
11	0	0	0	01	1h
11	0	0	1	01	1h
11	0	1	0	01	1h
11	0	1	1	01	1h
11	1	0	0	01	1h
11	1	0	1	01	1h
11	1	1	0	01	1h
11	1	1	1	01	1h

La ROM del cálculo de las salidas tiene direcciones de dos bits (el código de estado) y contenidos de 5 bits para las salidas:

Posición de memoria	Salidas			Codificación hexadecimal
	s_D	s_T	error	
00	00	00	0	00h
01	01	10	0	0Ch
10	10	01	0	12h
11	00	00	1	01h

El circuito que usa dos ROM es el siguiente:





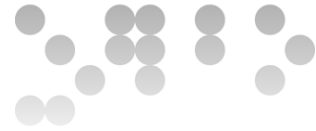
Con esta última versión se usan menos bits de memoria ROM: de 224 (32x7) con una única ROM a 84 (32x2 + 4x5).

- d) [20%] Implementad el circuito completo de la EFSM de la fig. 2: Construid el camino de datos usando las puertas lógicas y los bloques necesarios, e incorporad la unidad de control obtenida en el apartado anterior. Indicad y razonad, para cada bus, su dimensión en bits teniendo en cuenta que la señal de entrada N representa un valor sin signo (natural) de 3 bits.

Nota: Tenéis el ejercicio disponible en VerilCIRC. Tened en cuenta que VerilCIRC no soporta la verificación de subcircuitos diseñados por el usuario. Por lo tanto, tendréis que copiar en vuestra solución el circuito de la unidad de control que hayáis diseñado en el apartado anterior.

Por un lado, el camino de datos debe ocuparse de generar las señales necesarias para la unidad de control. En este caso, hay que generar las señales ($T > 0$) y ($D = 0$). Por otro, el camino de datos también tiene que actualizar y almacenar el valor de las variables que se usen y generar los resultados de las operaciones que sean necesarias. Aquí se usan las variables T y D .

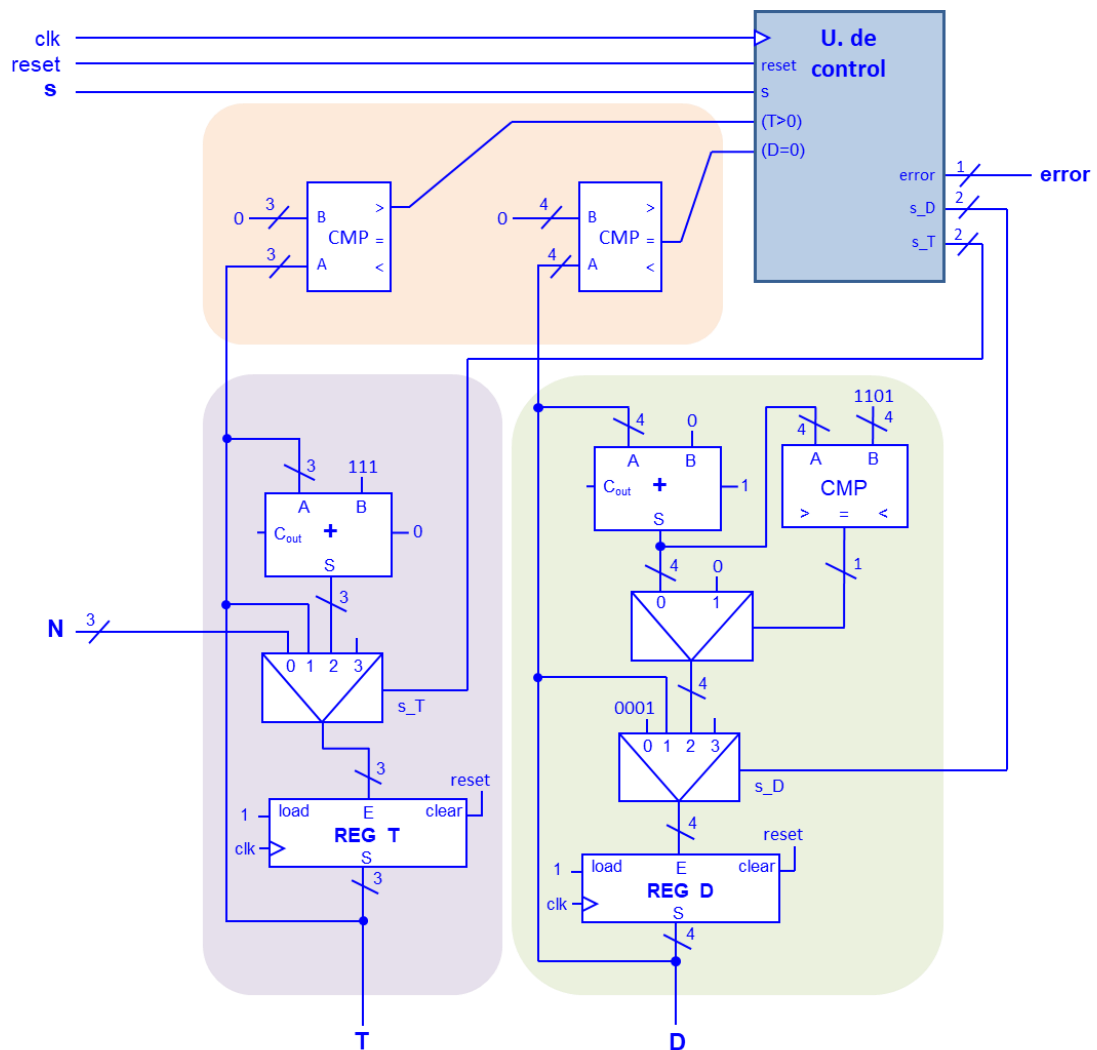
La entrada N es de 3 bits. Por lo tanto, la variable T también tiene que serlo, dado que el valor de N es el valor numérico mayor que tiene que guardar. Con un multiplexor controlado por s_T se determina el valor que llega a la entrada del registro que guarda el valor de T . En las líneas de datos de este multiplexor tendremos, la entrada N en la línea 0, el valor de T en la línea 1 y la salida de un sumador que se encarga de hacer la operación $T-1$ en la línea 2. El resultado de la operación $T-1$ se consigue sumando a T el valor 111, dado que esta es la representación del valor -1 en complemento a 2.

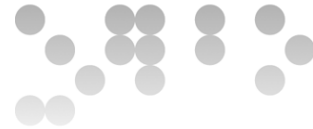


Del grafo podemos deducir que la variable D puede llegar a tener el valor 12, dado que se aplica $\text{mod } 13$ al incremento. Por lo tanto, son necesarios 4 bits para D . Como en el caso anterior, usaremos un multiplexor, aquí controlado por la señal s_D , para determinar el valor que llega a la entrada del registro que guarda el valor de D . En la línea 0 del multiplexor tendremos el valor 1, en la línea 1 el valor de D y en la línea 2 el resultado de la operación $(D+1) \text{ mod } 13$.

Con la operación $(D+1) \text{ mod } 13$ el valor de D se incrementa en una unidad. Aplicar $\text{mod } 13$ a este incremento quiere decir que el resultado de incrementar el valor 12 tiene que ser 0. Para conseguirlo, podemos usar un sumador para obtener $D+1$ y comparar el resultado de este sumador con el valor 13. El resultado de la comparación permitirá controlar un multiplexor que determinará si a la entrada del registro que guarda D tiene que llegar la salida del sumador o bien el valor 0.

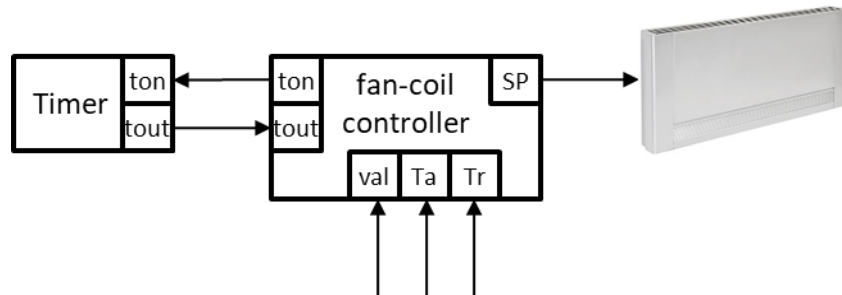
Con todo esto, el circuito completo para la EFSM de la fig. 2 queda de la forma siguiente:





SEGUNDA PARTE [35%]

Se tiene que diseñar un controlador para un *fan-coil* de calefacción:



(imagen *fan-coil*:
<https://www.dcasamayor.es/producto/smart-fan-coil-panasonic>)

Este dispositivo consta de las señales de entrada y salida siguientes:

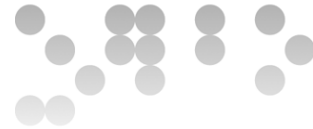
- La entrada *val* que habilita el funcionamiento del *fan-coil* cuando es 1 y lo inhabilita cuando es 0,
- La entrada *Ta* que indica la temperatura ambiente en grados Celsius con una precisión de décimas de grado,
- Una entrada *Tr* con la temperatura en grados Celsius que se quiere mantener en el espacio donde está instalado el *fan-coil*,
- La entrada *tout* que recibe un 1 cuando el temporizador finaliza la cuenta.
- La salida *ton*, que inicializa y pone en marcha el temporizador cada vez que cambia de 0 a 1.
- La salida *SP* que, con su valor, establece la velocidad a la que funciona el ventilador del *fan-coil*: parado con 0, velocidad baja con 1, velocidad media con 2 y velocidad máxima con 3.

Mientras la señal *val* tiene el valor 0, el *fan-coil* está parado. Cuando es 1 el funcionamiento del *fan-coil* está habilitado. El *fan-coil* se tiene que parar de manera inmediata, sin esperas, cuando la señal *val* toma el valor 0. La forma de parar el *fan-coil* es dar el valor 0 a *SP*.

Cuando el funcionamiento está habilitado, el controlador hace que la velocidad del ventilador sea la máxima si la temperatura ambiente (*Ta*) es más de 2 grados inferior a la temperatura de referencia (*Tr*); fija la velocidad media cuando la temperatura ambiente está entre 1 y 2 grados por debajo de la temperatura de referencia; y mantiene la velocidad baja cuando la diferencia entre la temperatura ambiente y la de referencia es inferior a 1 grado.

Para evitar situaciones en que el ventilador se esté parando y poniendo en marcha continuamente se establecen dos mecanismos para reducir el número de cambios de velocidad. Por un lado, la salida *SP* sólo puede cambiar su valor cada 60 segundos, excepto si *val* toma el valor 0, situación en la que se tiene que parar inmediatamente. Para conseguirlo, se usa un temporizador que se inicializa y se pone en marcha cuando la señal *ton* pasa de 0 a 1. Transcurridos 60 segundos el temporizador pone la señal *tout* a 1 durante 1 ciclo de reloj.

Por otro lado, se establece una histéresis: el *fan-coil* no se para hasta que la temperatura ambiente llega a superar la temperatura de referencia en 0,5 grados, y no se vuelve a poner en marcha hasta que la temperatura ambiente está más de 0,5



grados por debajo de la temperatura de referencia. Por lo tanto, cuando se pone en marcha el sistema ($val=1$) el ventilador del *fan-coil* se mantiene en marcha hasta que se supera en más de 0,5 grados la temperatura de referencia.

Diseñad la ASM correspondiente al comportamiento descrito para este controlador.

El propósito del controlador es establecer la velocidad del ventilador del *fan-coil*, es decir, el valor de la señal SP . Siempre que la señal de habilitación, val , sea 0, SP debe que tener el valor 0. Por lo tanto, la ASM tiene que contemplar la evaluación de esta señal en cada transición de estado, dado que la parada tiene que ser inmediata.

Si la señal val es 1, hay que evaluar la diferencia entre la temperatura ambiente (T_a) y la temperatura de referencia (T_r). Cuando la diferencia es superior a 2, SP tiene que tomar el valor 3; cuando se encuentra entre 1 y 2 tiene que tomar el valor 2; cuando la diferencia es inferior a 1 tiene que tomar el valor 1; y tiene que ser 0, es decir, parado, cuando se supera la temperatura de referencia en más de $\frac{1}{2}$ grado. Tendremos, pues, los estados HI, MID, LOW y STOP para asignar el valor correspondiente a SP .

Además de establecer el valor de SP , en cada uno de estos estados tenemos que poner en marcha el temporizador, $ton=1$. Hasta que el temporizador no finalice la cuenta, $tout=1$, no se podrá volver a cambiar el valor de SP , salvo que val pase a 0. Por lo tanto, una vez asignado el valor de SP hay que comprobar la señal val y, si es 1, comprobar el valor de $tout$. El valor de SP debe mantenerse mientras $tout$ sea 0. Para reducir el número de estados necesarios haremos que SP sea una variable.

Por otro lado, este controlador debe tener en cuenta una histéresis. El ventilador se tiene que mantener en marcha hasta que se supera la temperatura de referencia en más de $\frac{1}{2}$ grado. En ese momento, el ventilador se para y no se vuelve a poner en marcha hasta que la temperatura ambiente es como mínimo $\frac{1}{2}$ grado inferior a la de referencia. Esto quiere decir que en el intervalo que va desde medio grado por debajo de la referencia hasta medio grado por encima, el ventilador estará parado si se ha llegado a superar la referencia en más de medio grado y estará en marcha, a velocidad 1, si todavía no se ha llegado. Por lo tanto, hay que recordar si se ha superado este umbral. Para hacerlo, usaremos una variable, hi , que se pone a 1 cuando se supera el umbral y que se mantiene en ese valor hasta que la temperatura ambiente llega a estar más de medio grado por debajo de la referencia.

La figura siguiente muestra la ASM resultante. En este grafo se diferencia el estado INI del estado OFF para garantizar que el valor de la variable hi es el correcto cuando se evalúa su valor, dado que las variables toman el valor que se asigna en un estado en el ciclo de reloj siguiente.

